

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

Departamento de Computación, Electrónica y Mecatrónica

Reporte de práctica

Práctica 2: Unidad Aritmética Lógica (ALU) RISC-V de 32 bits

Materia	P26-LIS2022-1 Arquitecturas Computacionales
Profesor	Dr. Omar Guillén Fernández
Sección	2
Periodo	Primavera 2026

Integrante

Robbie Nicolás Curioso de Salazar 186028 robbie.curiosodr@udlap.mx



1. Introducción

La unidad aritmética lógica (ALU) es el bloque combinacional encargado de evaluar las operaciones que indica una instrucción dentro de la ruta de datos de un procesador. En arquitecturas RISC-V, la ALU del subconjunto RV32I debe atender suma y resta con y sin signo, operaciones lógicas bit a bit, corrimientos lógicos y aritméticos, y comparaciones que generan un resultado booleano de 32 bits.

En esta práctica se diseñó una ALU de 32 bits con dieciséis operaciones tomadas de RV32I y se generaron las cuatro banderas de estado (*Zero*, *CarryOut*, *Overflow* y *Sign*) requeridas para acoplarse a una unidad de control. La verificación se realizó con un banco de pruebas que cubre todas las operaciones más casos borde (cero, valores extremos y desbordamientos), simulado en *Active-HDL*.

2. Objetivos

- Implementar en VHDL una ALU de 32 bits con las dieciséis operaciones de RV32I solicitadas en la guía: ADD, SUB, AND, OR, XOR, NOR, SLL, SRL, SRA, SLLI, SRLI, SRAI, SLT, SLTU, LUI y AUIPC.
- Generar correctamente las banderas *Zero*, *CarryOut*, *Overflow* y *Sign*, asegurando que las dos primeras se calculen para suma y resta y se mantengan en cero para operaciones lógicas y de corrimiento.
- Construir un banco de pruebas que aplique al menos un vector por operación y casos borde representativos.
- Validar el diseño en *Active-HDL* mediante diagramas de tiempo y comparar contra los resultados esperados.

3. Materiales y herramientas

La práctica se desarrolla a nivel de simulación funcional, por lo que no requiere hardware adicional al equipo de cómputo:

- **Software:** Active-HDL Student Edition para compilación y simulación.
- **Lenguaje:** VHDL, con uso de las bibliotecas `ieee.std_logic_1164` y `ieee.numeric_std`.

- **Documentación:** guía de la práctica y especificación de RISC-V (*Volume I: Unprivileged Specification*, RV32I).

4. Marco teórico

Un procesador ejecuta una instrucción descomponiéndola en pasos de búsqueda, decodificación y ejecución. La etapa de ejecución delega en la ALU el cálculo del valor que se escribirá en un registro o que se usará como dirección. Para cubrir el conjunto base RV32I, una ALU debe soportar tres familias de operaciones:

- **Aritméticas.** ADD y SUB con signo en complemento a dos. La resta se obtiene como $A + (\sim B + 1)$. En sumadores extendidos, el bit de salida adicional sirve para detectar el acarreo (*CarryOut*); el desbordamiento con signo se detecta comparando los signos de los operandos contra el del resultado.
- **Lógicas bit a bit.** AND, OR, XOR y NOR. No requieren acarreo ni desbordamiento, por lo que esas dos banderas se fijan en cero.
- **Corrimientos.** SLL (lógico a la izquierda), SRL (lógico a la derecha) y SRA (aritmético a la derecha, preserva el signo). RV32I especifica que la cantidad de corrimiento se codifica en cinco bits, por lo que en la ALU se toma $B[4 : 0]$ aunque B sea un vector de 32 bits. Las versiones inmediatas (SLLI, SRLI, SRAI) son funcionalmente idénticas a las anteriores y se diferencian a nivel de codificación de instrucción.
- **Comparaciones.** SLT y SLTU comparan dos operandos como con signo y sin signo, respectivamente, y devuelven un valor de 32 bits que vale 0x00000001 cuando $A < B$ y 0x00000000 en caso contrario.
- **Inmediatos superiores.** LUI carga un inmediato desplazado 12 lugares a la izquierda, lo que coloca un valor de 20 bits en la mitad alta del resultado. AUIPC suma ese mismo inmediato desplazado al valor del Program Counter; en la ALU diseñada se adopta la convención de pasar el PC por el operando A cuando `ALUControl=1110`.

Las banderas que produce la ALU resumen el estado del resultado: *Zero* se activa si el resultado es cero, *Sign* replica el bit más significativo (signo en complemento a dos), *CarryOut* corresponde al acarreo de salida en suma sin signo y al préstamo en resta sin signo, y *Overflow* indica desbordamiento con signo en operaciones aritméticas.

5. Diseño y procedimiento

5.1. Arquitectura de la ALU

La entidad `alu32` expone tres entradas (`opa[31:0]`, `opb[31:0]` y `alu_ctrl[3:0]`) y cinco salidas (`result[31:0]`, `carry_out`, `overflow`, `zero` y `sign`). El ancho de datos se parametriza con un `generic` llamado `data_width` que se fija en 32 bits para esta práctica. La selección de operación se realiza con un `case` sobre `alu_ctrl`, donde cada uno de los dieciséis valores se asoció con una constante simbólica para mejorar la legibilidad.

Cuadro 1: Codificación de `alu_ctrl` y operación correspondiente.

<code>alu_ctrl</code>	Operación	Descripción
0000	ADD	Suma con signo
0001	SUB	Resta con signo
0010	AND	AND lógico bit a bit
0011	OR	OR lógico bit a bit
0100	XOR	XOR lógico bit a bit
0101	SLL	Corrimiento lógico a la izquierda
0110	SRL	Corrimiento lógico a la derecha
0111	SRA	Corrimiento aritmético a la derecha
1000	SLT	Set on less than (con signo)
1001	SLTU	Set on less than (sin signo)
1010	SLLI	Corrimiento lógico izquierda con inmediato
1011	SRLI	Corrimiento lógico derecha con inmediato
1100	SRAI	Corrimiento aritmético derecha con inmediato
1101	LUI	Load Upper Immediate ($B \ll 12$)
1110	AUIPC	Add Upper Immediate to PC ($A + (B \ll 12)$)
1111	NOR	NOR lógico bit a bit

5.2. Generación de banderas

Las banderas *Zero* y *Sign* se obtienen al final del proceso, sin importar la operación: *Zero* compara el resultado contra un vector de ceros y *Sign* toma el bit más significativo. Por defecto, *CarryOut* y *Overflow* se mantienen en cero y solo se sobrescriben en **ADD** y **SUB**.

Para obtener *CarryOut* en suma se extiende cada operando con un cero a la izquierda for-

mando vectores de 33 bits. El bit más significativo del resultado de 33 bits es el acarreo. Para la resta se aplica la misma extensión y se interpreta el bit superior como préstamo: vale 1 cuando $A < B$ en interpretación sin signo. El criterio de desbordamiento se evalúa por comparación de signos:

- En suma hay *Overflow* cuando los signos de A y B coinciden y el del resultado es opuesto.
- En resta hay *Overflow* cuando los signos de A y B son distintos y el signo del resultado difiere del signo de A .

5.3. Banco de pruebas

El testbench `tb_alu32` aplica veintitrés vectores de estímulo cada 10.000 ns cubriendo las dieciséis operaciones más casos borde. La Tabla 2 muestra el conjunto completo, con los valores aplicados, el resultado esperado y las banderas que deben activarse. Este conjunto incluye desbordamientos en suma y en resta, valor cero en operandos, máximo positivo, mínimo negativo y patrones complementarios para verificar las operaciones lógicas.

Cuadro 2: Vectores aplicados en el banco de pruebas y resultados esperados.

#	ctrl	Operación	A	B	Resultado	Banderas
1	0000	ADD	00000005	00000003	00000008	–
2	0000	ADD	7FFFFFFF	00000001	80000000	OF=1, S=1
3	0000	ADD	FFFFFFFF	00000001	00000000	CO=1, Z=1
4	0001	SUB	0000000A	00000003	00000007	–
5	0001	SUB	00000003	0000000A	FFFFFFF7	CO=1, S=1
6	0001	SUB	80000000	00000001	7FFFFFFF	OF=1
7	0010	AND	FF00FF00	0F0F0F0F	0F000F00	–
8	0011	OR	0000FFFF	FFFF0000	FFFFFFFF	S=1
9	0100	XOR	AAAA5555	5555AAAA	FFFFFFFF	S=1
10	1111	NOR	00FF00FF	FF00FF00	00000000	Z=1
11	0101	SLL	00000001	00000004	00000010	–
12	0110	SRL	80000000	00000004	08000000	–
13	0111	SRA	80000000	00000004	F8000000	S=1
14	1010	SLLI	00000001	00000008	00000100	–
15	1011	SRLI	FF000000	00000004	0FF00000	–
16	1100	SRAI	FF000000	00000004	FFF00000	S=1
17	1000	SLT	FFFFFFFF	00000001	00000001	–
18	1000	SLT	00000005	00000003	00000000	Z=1
19	1001	SLTU	00000001	FFFFFFFF	00000001	–
20	1001	SLTU	00000005	00000003	00000000	Z=1
21	1101	LUI	00000000	000ABCDE	ABCDE000	S=1
22	1110	AUIPC	00001000	00000010	00011000	–
23	0000	ADD	00000000	00000000	00000000	Z=1

5.4. Flujo de simulación en Active-HDL

Para cada simulación se siguió el mismo procedimiento:

1. Crear el espacio de trabajo y agregar al diseño los archivos `alu32.vhd` y `tb_alu32.vhd`.
2. Compilar primero la entidad y después el testbench, verificando que no se generen errores ni advertencias.
3. Inicializar la simulación seleccionando `tb_alu32` como nivel superior.

4. Abrir un *waveform* y agregar las ocho señales de interés (*opa_tb*, *opb_tb*, *alu_ctrl_tb*, *result_tb*, *carry_out_tb*, *overflow_tb*, *zero_tb* y *sign_tb*).
5. Ajustar el formato de los vectores: hexadecimal para los operandos y el resultado, binario para *alu_ctrl*.
6. Ejecutar *Run For* con duración de 250.000 ns y aplicar *Zoom to Fit*.

6. Resultados y discusión

La Figura 1 presenta la simulación completa de los 23 vectores. Cada cambio de valor en *alu_ctrl_tb* delimita una operación distinta y permite seguir la secuencia de pruebas a lo largo del eje de tiempo. En las subsecciones siguientes se analizan los resultados por familia de operación con un acercamiento al rango temporal correspondiente.

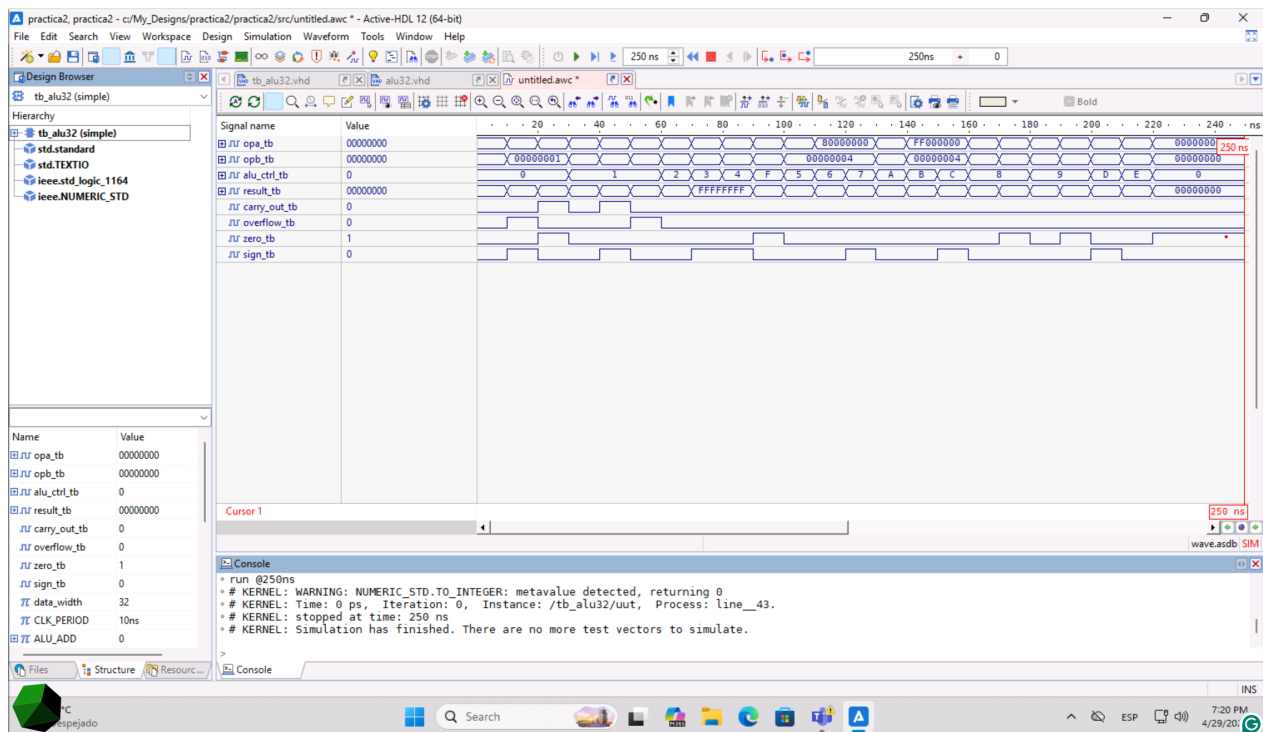


Figura 1: Vista panorámica de la simulación: 23 vectores ejecutados de 0.000 ns a 230.000 ns.

6.1. Operaciones aritméticas

La Figura 2 muestra los seis casos aritméticos. En el primer vector se realiza una suma sencilla $5+3=8$ sin activar ninguna bandera más allá de las predeterminadas. En el segundo,

7FFFFFFF + 1 produce 80000000, que en complemento a dos corresponde al mínimo entero negativo: *Overflow* pasa a 1 y *Sign* también, ya que el resultado tiene el bit más significativo activo. El tercer caso es un *wrap-around* sin signo: FFFFFFFF + 1 se desborda a cero, lo que activa simultáneamente *CarryOut* y *Zero*.

Para la resta, el caso $10 - 3$ no genera banderas extras. Cuando los operandos se invierten ($3 - 10$) el resultado en complemento a dos es **FFFFFF7** (es decir, -7): *Sign* se activa y *CarryOut* también, ya que la condición sin signo $A < B$ se interpreta como préstamo. El último caso, $80000000 - 1$, ilustra el desbordamiento clásico al restar uno al mínimo negativo: el resultado pasa a ser positivo (**7FFFFFFF**) y *Overflow* se enciende.

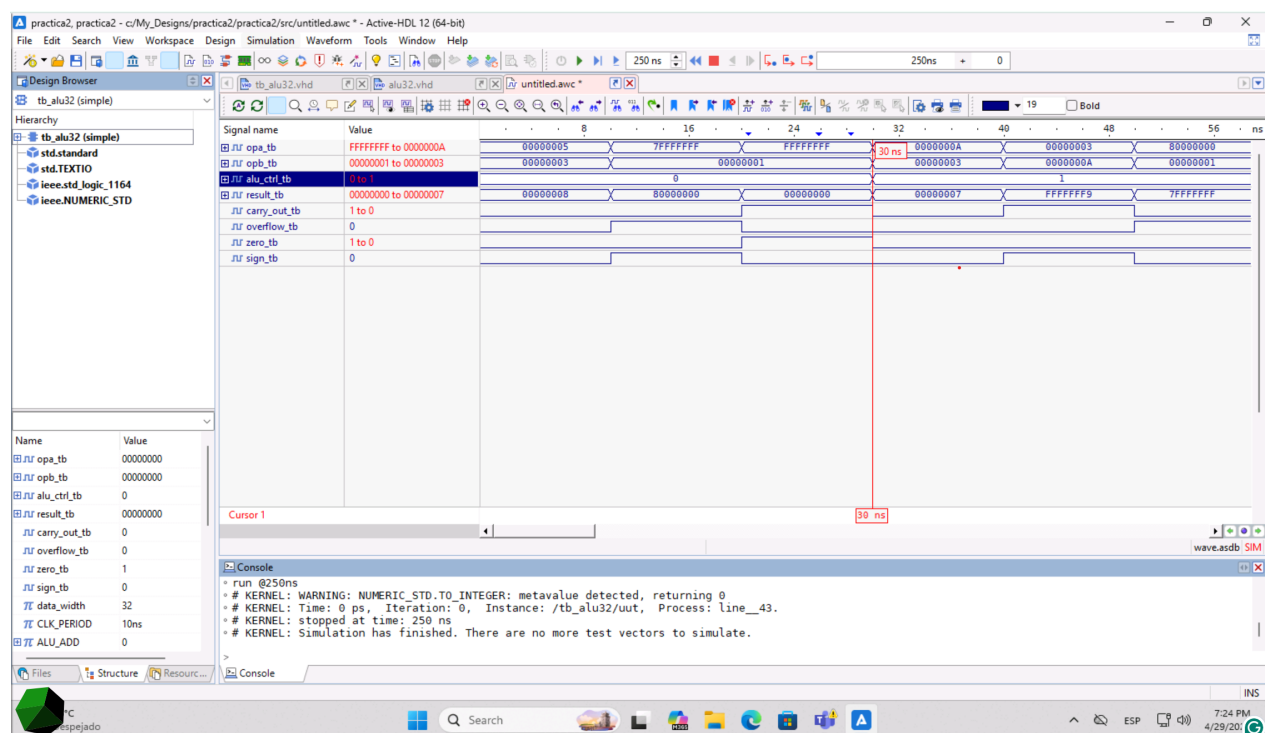


Figura 2: Operaciones aritméticas **ADD** y **SUB** (0.000 ns a 60.000 ns).

6.2. Operaciones lógicas

En la Figura 3 se observan los cuatro casos lógicos. La operación AND entre FF00FF00 y 0F0F0F0F deja en cada nibble únicamente los bits comunes, dando 0F000F00. La OR entre dos máscaras complementarias en mitad alta y baja produce todos los bits en uno (FFFFFFFF), con *Sign* activo. La XOR sobre patrones AAAA5555 y 5555AAAA también devuelve FFFFFFFFFF, ya que las posiciones de los bits están exactamente intercambiadas. Por último, la NOR entre 00FF00FF y FF00FF00 produce 00000000 porque los dos operandos cubren todas las posicio-

nes; este caso ejercita la bandera *Zero*.

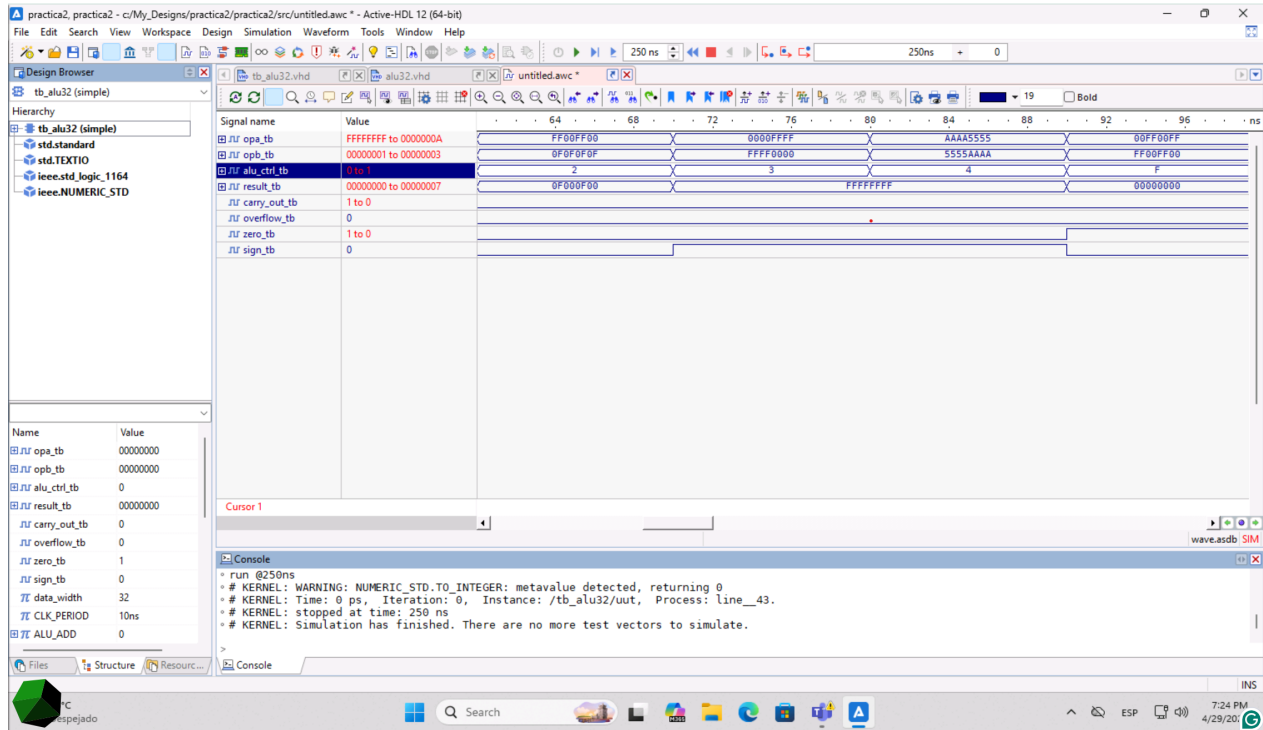


Figura 3: Operaciones lógicas AND, OR, XOR y NOR (60.000 ns a 100.000 ns).

6.3. Corrimientos por registro

La Figura 4 ilustra los tres corrimientos cuando la cantidad se toma de $B[4 : 0]$. Para SLL con $A = 00000001$ y un corrimiento de cuatro posiciones, el resultado es 00000010 . Con 80000000 y la misma cantidad, SRL desplaza ceros desde la izquierda y produce 08000000 , mientras que SRA replica el bit de signo y produce $F8000000$. La diferencia entre las dos últimas operaciones es la evidencia clave del corrimiento aritmético: SRA preserva el signo y mantiene la bandera *Sign* activa, lo cual es esperable cuando el operando es negativo.

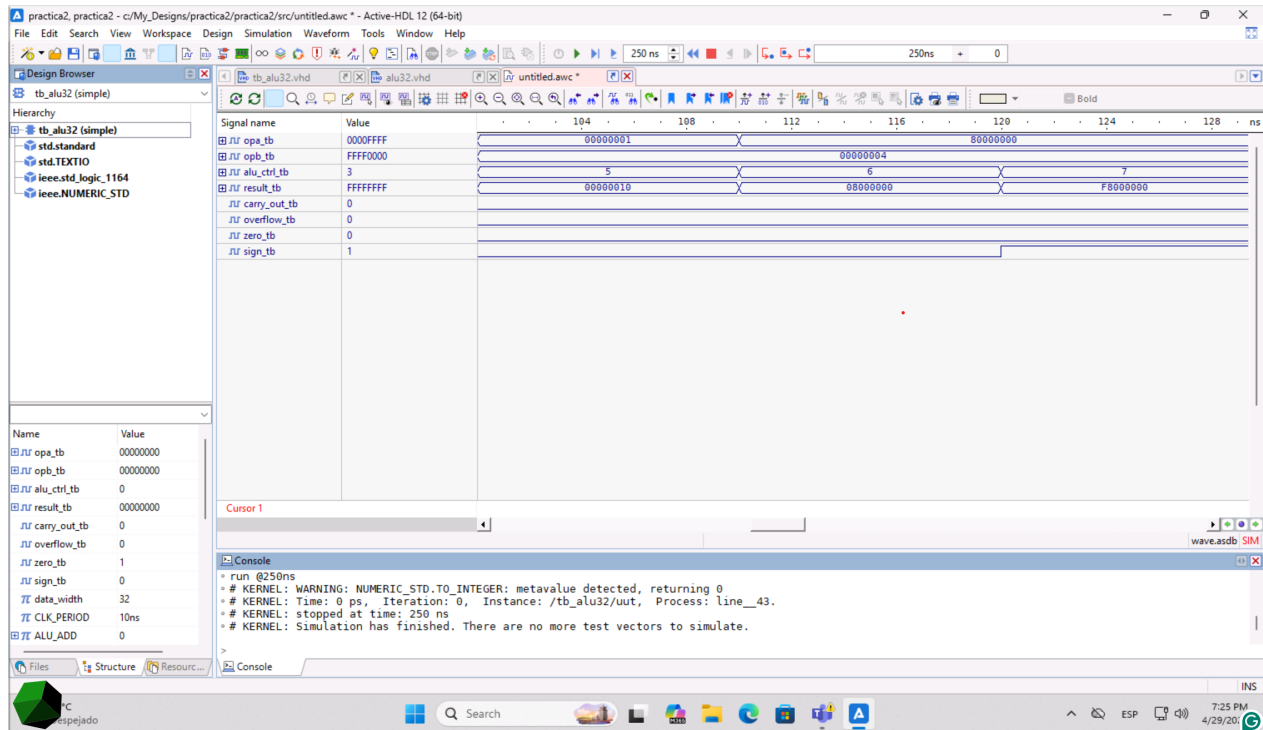


Figura 4: Corrimientos SLL, SRL y SRA (100.000 ns a 130.000 ns).

6.4. Corrimientos con inmediato

A nivel de hardware, las versiones SLLI, SRLI y SRAI comparten lógica con sus contrapartes; la única diferencia conceptual es el origen del operando B , que en el caso inmediato proviene del campo de instrucción y se trunca a cinco bits. En la Figura 5 se aplican $A = 00000001$ con desplazamiento de ocho (00000100), y $A = FF000000$ con desplazamiento de cuatro para SRLI (0FF00000) y SRAI (FFF00000). Nuevamente la diferencia entre las dos últimas confirma la propagación del bit de signo en el corrimiento aritmético.

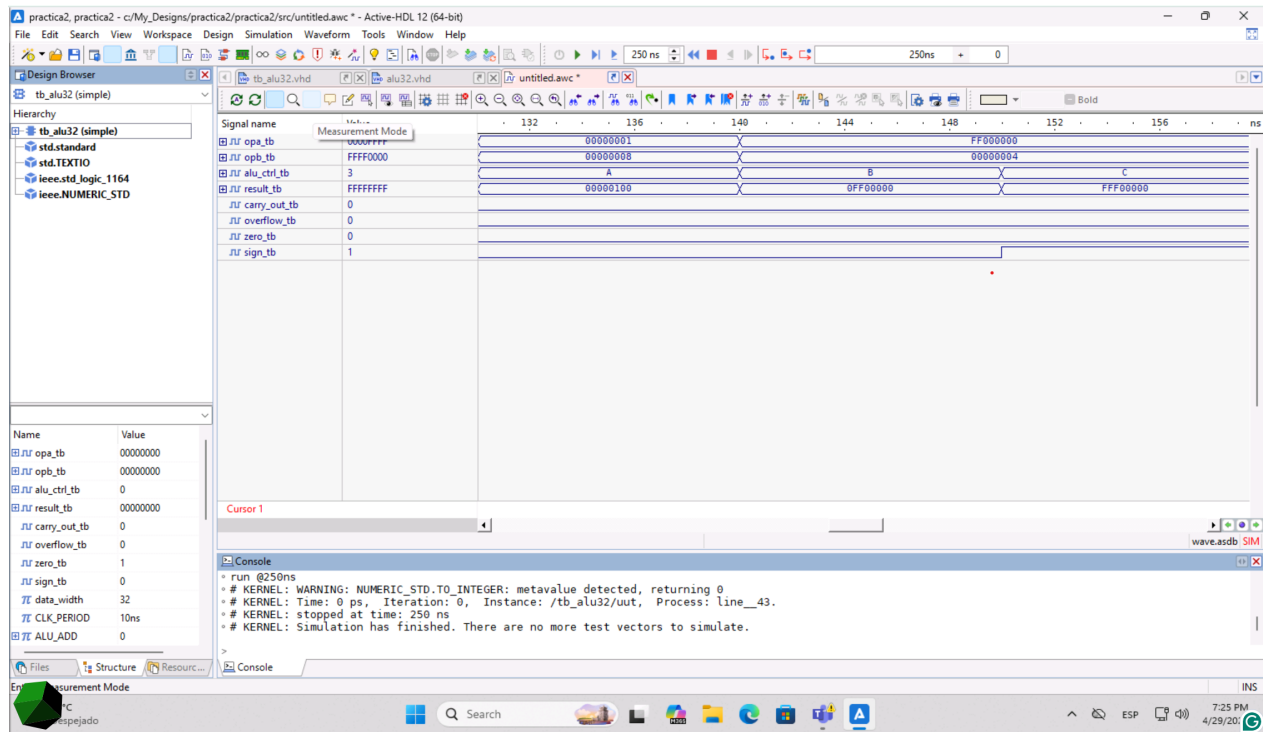


Figura 5: Corrimientos con inmediato SLLI, SRLI y SRAI (130.000 ns a 160.000 ns).

6.5. Comparaciones

En la Figura 6 se comparan los casos con signo y sin signo. Para SLT, $A = \text{FFFFFFFF}$ se interpreta como -1 y $B = 1$ como uno positivo; al ser $-1 < 1$, el resultado es 00000001 . El segundo caso, $5 < 3$, es falso y el resultado vale cero. Para SLTU la interpretación es sin signo: $00000001 < \text{FFFFFFFF}$ es verdadero (uno frente al máximo no firmado), por lo que el resultado vuelve a ser 00000001 ; en el último caso, $5 < 3$ sin signo también es falso. La comparación entre los dos casos $A = \text{FFFFFFFF}$, $B = 1$ resume bien la diferencia entre SLT y SLTU: el mismo par de operandos produce 1 en la versión con signo y 0 en la versión sin signo.

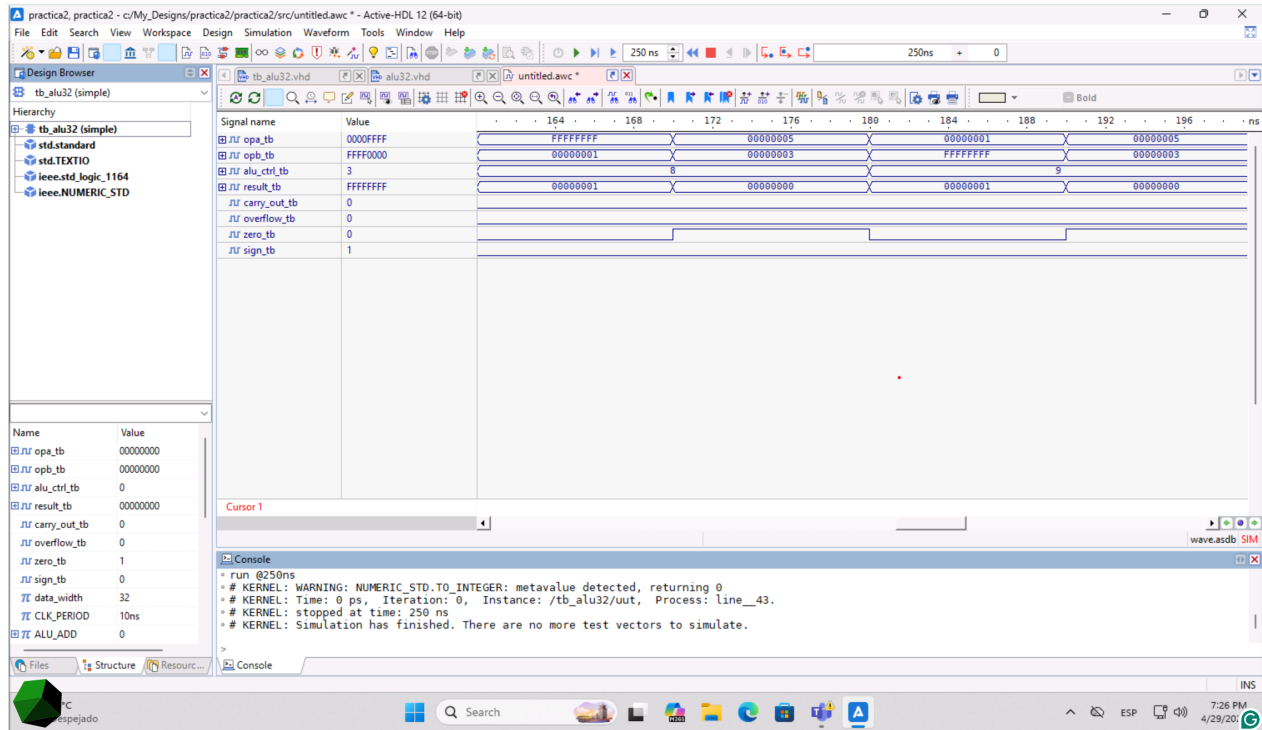


Figura 6: Comparaciones SLT y SLTU (160.000 ns a 200.000 ns).

6.6. LUI, AUIPC y caso de bandera Zero

La Figura 7 cubre los últimos tres vectores. La operación LUI con $B = 000ABCDE$ desplaza B doce posiciones a la izquierda y obtiene $ABCDE000$, que coloca el inmediato en los 20 bits superiores del resultado. AUIPC suma al PC (operando $A = 00001000$) el inmediato $B = 00000010$ ya desplazado, dando 00011000 . Finalmente, el último vector aplica ADD con ambos operandos en cero para verificar la bandera *Zero*: el resultado es 00000000 y *Zero* se enciende.

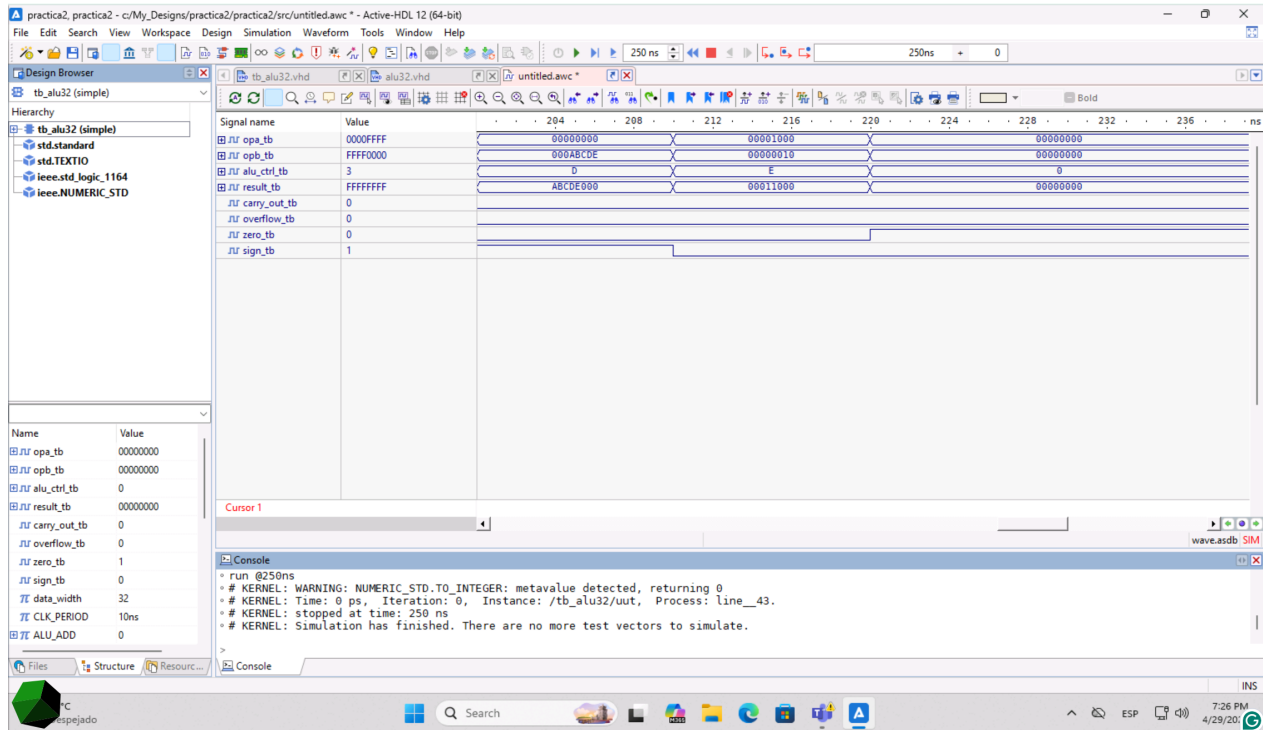


Figura 7: LUI, AUIPC y verificación de la bandera *Zero* (200.000 ns a 240.000 ns).

7. Cuestionario / Actividades complementarias

7.1. ¿Por qué la cantidad de corrimiento usa solo $B[4 : 0]$?

Porque en RV32I los registros son de 32 bits y un corrimiento mayor a 31 no tiene sentido aritmético: desplazaría todo el contenido hasta dejar solo ceros (o el bit de signo, en el caso aritmético). Cinco bits permiten codificar exactamente los valores 0 a 31. Tomar más bits duplicaría información sin agregar comportamiento, además de ser inconsistente con la codificación del campo inmediato en las instrucciones SLLI, SRLI y SRAI.

7.2. ¿Cuál es la diferencia entre SLT y SLTU?

Ambas devuelven 0x00000001 cuando $A < B$ y 0x00000000 en caso contrario, pero la interpretación de los operandos cambia. SLT los toma como enteros con signo en complemento a dos, por lo que FFFFFFFF es -1 y resulta menor que cualquier número positivo. SLTU los interpreta sin signo, donde FFFFFFFF es el valor máximo posible y resulta mayor que cualquier otro operando salvo él mismo.

7.3. ¿Cómo se detecta el desbordamiento en suma y resta con signo?

En suma, hay desbordamiento cuando los dos operandos tienen el mismo signo y el resultado tiene signo opuesto: dos positivos no pueden sumar un resultado negativo en aritmética correcta. En resta, hay desbordamiento cuando los signos de los operandos son distintos y el signo del resultado no coincide con el signo de A . La fórmula equivalente $\text{Overflow} = A_{31} \oplus B_{31} \oplus R_{31}$ funciona para suma; para resta hay que considerar $\sim B$ o aplicar la condición directa de signos.

7.4. ¿Por qué LUI usa un corrimiento de 12 posiciones?

Porque la codificación del inmediato superior en RV32I dedica 20 bits al campo inmediato, y este valor representa los 20 bits más significativos del resultado de 32 bits. Desplazar el inmediato 12 posiciones a la izquierda lo coloca en la mitad alta y deja en cero los 12 bits inferiores, que típicamente se completan con un ADDI posterior para construir constantes de 32 bits.

8. Buenas prácticas

Durante el desarrollo se aplicaron varias prácticas que conviene mantener en futuros diseños: usar la biblioteca `numeric_std` en lugar de `std_logic_arith` para que las conversiones entre `signed`, `unsigned` y `std_logic_vector` sean explícitas; declarar constantes simbólicas para los códigos de operación, lo cual facilita la lectura del `case`; e inicializar variables internas con valores por defecto al inicio del proceso para evitar advertencias de *metavalue* durante la simulación. En el banco de pruebas se separó cada vector con un `wait for CLK_PERIOD` para que las transiciones queden alineadas en la *waveform*, lo que simplifica la inspección visual.

9. Conclusiones

Se implementó y verificó una ALU de 32 bits con las dieciséis operaciones de RV32I solicitadas, junto con las cuatro banderas de estado requeridas. Las simulaciones en Active-HDL coincidieron con los valores esperados de la Tabla 2 en todos los casos: las operaciones aritméticas reportaron correctamente los desbordamientos con signo y los acarreos sin signo, las lógicas dejaron las banderas aritméticas en cero y solo afectaron *Zero* y *Sign*, los corrimientos

respetaron tanto la cantidad limitada a cinco bits como la diferencia entre extensión con cero y extensión con signo, y las comparaciones discriminaron correctamente entre interpretación con y sin signo.

La parte que requirió mayor atención fue la generación de *CarryOut* y *Overflow* en suma y resta. La versión inicial de la ALU dejaba ambos en cero, lo que hubiera ocultado errores en la unidad de control. Al extender los operandos a 33 bits y aplicar las reglas de signo, el comportamiento quedó alineado con la especificación. La convención de usar el operando *A* como Program Counter en AUIPC mantiene la interfaz simple y evita agregar un puerto adicional.

Como trabajo futuro se propone construir un banco de pruebas auto-verificable con `assert` en cada vector, lo que permitiría detectar regresiones automáticamente al ampliar el conjunto de operaciones (por ejemplo, hacia las extensiones M y A de RISC-V) sin depender de la inspección visual de los diagramas de tiempo.

10. Referencias

Referencias

- [1] O. Guillén Fernández. *Práctica 2: Unidad Aritmética Lógica (ALU) RISC-V de 32 bits*. Universidad de las Américas Puebla.
- [2] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Specification*. Disponible en: <https://riscv.org/technical/specifications/>
- [3] D. A. Patterson y J. L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2020.
- [4] IEEE. *IEEE Standard VHDL Language Reference Manual*. (Referencia general al estándar).
- [5] Aldec. *Active-HDL Documentation*. Disponible en: https://www.aldec.com/en/products/fpga_simulation/active_hdl

Repositorio del proyecto

El código fuente VHDL de la ALU, el banco de pruebas y las capturas de las formas de onda obtenidas en Active-HDL se encuentran disponibles en el siguiente repositorio público de GitHub:

<https://github.com/Robbienicur/Arquitecturas-Computacionales/tree/main/Practica-2>

La carpeta Practica-2 del repositorio contiene los archivos `alu32.vhd` y `tb_alu32.vhd`, las imágenes de la simulación y este mismo reporte en formato PDF.